

Data Analysis with Python 2024–2025

Parte 6: Machine Learning Types

Caderno de Exercícios

Tradução e Adaptação: University of Helsinki — MOOC.fi

Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024-2025*, oferecido pela **Universidade de Helsinque (MOOC.fi)**.

Conteúdo

1 Exercícios - Capítulo 6 (Clustering)

Exercício 6.5 – Clustering de Plantas (`plant_clustering`)

Sumário

- **Objetivo:** Aplicar o agrupamento K-Means no clássico conjunto de dados Iris com 3 clusters definidos.
- **Procedimento:**
 1. Crie a função `plant_clustering()` que carregue os dados (`load_iris`).
 2. Inicialize o classificador `KMeans(3, random_state=0)` e faça o ajuste aos dados de treinamento.
 3. Use a técnica de permutação de rótulos (`find_permutation`) introduzida na lição para realinhar os nomes arbitrários dos clusters aos números corretos baseando-se na moda do alvo correto.
- **Retorno:** A `accuracy_score` da correspondência entre os rótulos do k-means com permutações e o alvo biológico real.

Exercício 6.6 – Clusters Não Convexos (`nonconvex_clusters`)

Sumário

- **Objetivo:** Ler o arquivo `src/data.tsv`, processá-lo pelo DBSCAN variando o limiar de busca (`eps`) e exportar uma matriz de relatórios do modelo em formato pandas `DataFrame`.
- **Procedimento:**
 1. Realize um laço para executar instâncias do DBSCAN baseados nos hiperparâmetros gerados por `np.arange(0.05, 0.2, 0.05)`.
 2. **Importante:** O DBSCAN rotula anomalias/outliers isoladas como -1. Você deve adaptar a função `find_permutation` para não computá-los durante a contagem de acurácia.
 3. Caso o modelo crie um número final de agrupamentos não corrompidos que destoe da quantidade de rótulos originais dos dados (após excluir os outliers), marque a acurácia para aquela etapa como `np.nan`.
- **Retorno:** Um `DataFrame` cujas colunas devem ser estritamente: `["eps", "Score", "Clusters", "Outliers"]`.

Exercício 6.7 – Locais de Ligação Molecular (`binding_sites`)

Sumário

- **Objetivo:** Construir um fluxo usando o `AgglomerativeClustering` (agrupamento hierárquico) capaz de discernir conjuntos genéticos (Adenina, Citosina, Guanina e Timina) a partir do `data.seq`.
- **Parte 1 (`toint` e `get_features_and_labels`):** Escreva a função que converte o nucleotídeo (A, C, G, T) num escalar sequencial (0, 1, 2, 3) iterando sobre a string contida na coluna X do dataset para criar a sua matriz de *features*.
- **Parte 2 (`cluster_euclidean`):** Crie um agrupamento aglomerativo hierárquico, estipulando 2 clusters, métrica de afinidade *euclidiana* com critério de união *average* (ligação por média). Use a permutação para recuperar rótulos e retorne o *score*.
- **Parte 3 (`cluster_hamming`):** Substitua a forma base e agora aplique a contagem de instâncias pela *distância de Hamming*. Utilize a função explícita `pairwise_distances(..., metric='hamming')` sob a matriz principal, e crie a classe de agrupamento instruída agora para aceitar vetores pré-calculados (`affinity='precomputed'`). Retorne a acurácia.